A Three-Layer Caching Architecture for Low-Latency LLM Web Search on Commodity CPU Hardware

Ayushman Bhattacharya
MyceliAI AÜ
ayushman@myceli.ai

***Abstract*—The past two years have seen a proliferation of AI-powered search products-OpenAI's SearchGPT (now integrated into ChatGPT), Google's AI Overviews, and Perplexity's real-time answer engine-each offering LLM-synthesized answers grounded in live web results. For end users, these are transformative. For developers building on top of them, the economics are punishing: OpenAI's web search API costs $0.03–0.10 per query (base fee plus injected context tokens), Google's Gemini with grounding starts at $35/1K queries, and Perplexity's Sonar API runs $5–18/1K requests depending on tier [11]. We set out to build an open-source alternative that could search the web with LLMs at a fraction of the cost, using automated browser agents and open models-running entirely on CPU, without GPU acceleration. The system worked, but as users grew, so did the problems. Sessions lost context between turns. Semantically identical queries triggered redundant LLM calls. The same URLs were embedded over and over across sessions. On a resource-constrained 8-vCPU server, every wasted computation mattered.**

This paper tells the story of how we solved those problems. We present a three-layer caching architecture designed to sustain indefinitely long conversations without degradation: (1) a *Session Context Window* maintaining a rolling window of recent messages in Redis with automatic overflow to Huffman-compressed disk archives; (2) a *Semantic Query Cache* that catches rephrasings via cosine similarity on embedding vectors, eliminating redundant LLM invocations; and (3) a *URL Embedding Cache* that deduplicates embedding computations across sessions. Deployed on a single 8-vCPU Intel Cascade Lake server (2 GHz, 32 GB RAM) running 30 Hypercorn worker processes across three containerized replicas, the system achieves an 89.3% Redis-level cache hit rate with 0.1 ms read latency and just 1.38 MB of memory overhead. A background LRU eviction daemon transparently migrates idle sessions from Redis to disk and rehydrates them on demand, enabling conversations that can be resumed hours or days later with full context preserved. We describe the journey from a bare-bones search pipeline to a production system, detail the architecture and implementation of each caching layer, and evaluate the system's throughput, latency, and compression characteristics on commodity CPU hardware.

***Index Terms*—LLM caching architecture, AI-powered web search, semantic query deduplication, conversational session management, Redis multi-tier cache, Huffman compression, embedding reuse, retrieval-augmented generation, commodity CPU inference, cost-efficient search**

I. Introduction

A. The Problem: AI Search APIs Are Too Expensive

The past two years have brought a wave of AI-powered search products. OpenAI launched SearchGPT in late 2024 (now integrated into ChatGPT). Google added AI Overviews with Gemini grounding. Perplexity built a real-time answer engine with its Sonar API. For end users, these are impressive. For developers building on top of them, the economics tell a different story.

The API pricing across providers is steep. OpenAI charges $10–30 per thousand search calls as a base fee (varying by model and context tier), plus ∼8,000 tokens of injected web context billed at standard input rates. In practice, a single query costs $0.03–0.10 depending on the model used [9]. Developers on OpenAI's community forums reported bills 2–3× higher than expected, with some paying over $2 for just 21 searches (∼$0.10 each) [10]. Google's Gemini with grounding charges $35 per 1,000 grounded queries for the Pro model ($14/1K for Flash) [12]. Perplexity's Sonar API charges $5 per 1,000 requests for the base search tier, scaling to $18/1K for Sonar Pro [11]. For a search assistant handling thousands of queries per day, these per-query fees add up quickly.

We wanted to build something different: a search assistant that could browse the web, synthesize answers with sources, and carry on multi-turn conversations—but at a cost we could actually sustain.

B. The First Version: Raw Search + LLM

So we built our search assistant from scratch. The initial version was deliberately simple. Instead of paying per-query fees to a proprietary search API, we used automated headless browser agents to perform web searches directly-the same way a human would. The agents navigated search engines, extracted results, fetched full page content, and fed it to an open LLM for synthesis. There was no retrieval-augmented generation (RAG), no vector database, no caching layer. Just a search agent pool, a language model, and a pipeline connecting them.

It worked. The per-query cost dropped dramatically—from $0.03–0.10 with SearchGPT to fractions of a cent with open models and free search. But as users grew and conversations got longer, three problems emerged that threatened to erode every cost advantage we had built:

1) **"What did we just talk about?"** Users expected the system to remember context across turns. Storing entire conversation histories in memory did not scale across thousands of concurrent sessions. Without context, the

assistant repeated itself, missed follow-up nuances, and frustrated users.

2) **"Didn't we already answer this?"** Users frequently rephrased queries: "weather Tokyo" followed by "Tokyo weather forecast." Each rephrasing triggered a full pipeline execution—search agents, page fetches, LLM synthesis—even though the answer was already computed seconds ago.
3) **"We already embedded this URL."** As we added RAG capabilities to improve answer quality, the same popular URLs were fetched and embedded by multiple sessions. Computing a 384-dimensional embedding ($\sim$200 ms per URL) redundantly across sessions wasted the compute budget we had fought so hard to minimize.

C. The Solution: A Three-Tier Cache Architecture

Each of these problems had partial solutions in the ecosystem. LangChain [1] offered in-memory conversation buffers. GPTCache [2] provided semantic caching for LLM responses. But nothing unified all three concerns into a single, lightweight system we could integrate into our existing pipeline without adding heavyweight infrastructure.

So we built a three-layer caching architecture that grew organically out of the problems we faced in production. Each layer was created due to a specific pain point:

- **Layer 1: Session Context Window.** A rolling window of the 20 most recent messages in Redis, with automatic overflow to Huffman-compressed disk archives.
- **Layer 2: Semantic Query Cache.** Catches rephrasings by comparing query embeddings via cosine similarity ($\tau \geq 0.90$), returning cached LLM responses and skipping the entire pipeline. Created from watching the same query hit our search agents dozens of times in slightly different words.
- **Layer 3: URL Embedding Cache.** A global, cross-session store of pre-computed embedding vectors with 24-hour TTL. Created from profiling and discovering that a significant fraction of embedding computations were redundant across concurrent sessions.

The system also introduced a **two-tier hybrid storage model**: Redis for hot data ($\sim$1 ms reads) and Huffman-compressed disk archives for cold data, connected by a background LRU eviction daemon. A **custom binary archive format** with a 24-byte header achieves 45–67% compression on conversation text using zero native dependencies. All tunables are consolidated into a single configuration object with environment variable support, making the system deployable across different environments without code changes.

In production, this architecture serves an 89.3% cache hit rate with just 1.38 MB of Redis memory, enabling conversations that persist indefinitely across sessions, server restarts, and horizontal scaling events.

The remainder of this paper follows the arc of this journey. Section II presents the architecture. Section III explains the key design decisions. Section IV details the implementation. Section V evaluates production performance. Section VI surveys related work. Section VII discusses limitations, and Section VIII concludes.

II. Architecture

A. Overview

With the problems identified, the architecture took shape around a simple principle: each caching concern gets its own logical partition within a single Redis [8] instance (DB 0 for semantic query cache, DB 1 for URL embeddings, DB 2 for session context), and a single coordinator process ties them together. Figure 1 illustrates the complete data flow when a user message arrives.

B. Layer 1: Session Context Window (Redis DB 2)

Users expected the assistant to remember what they had said two turns ago. The Session Context Window maintains a rolling window of the k most recent messages (default k=20) for each session. Messages are stored as individual Redis keys with TTL, and an ordered list tracks message insertion order.

When a new message arrives, it is pushed to the head of the Redis list. If the list exceeds k entries, the oldest message is popped, serialized, and appended to a Huffman-compressed disk archive. This ensures Redis memory usage remains bounded at $O(k)$ per session regardless of conversation length.

When the application requests context and Redis is empty (e.g., after LRU eviction), the system transparently re-hydrates by loading the last k messages from the disk archive back into Redis. If Redis is entirely unavailable, the system falls back to disk-only reads to ensure no downtime.

C. Layer 2: Semantic Query Cache (Redis DB 0)

Users do not type the same query twice-they rephrase it. "What's the weather in Tokyo" becomes "Tokyo weather forecast" becomes "Tokyo temperature today." Each variation triggered a full pipeline run: search agents launched, pages fetched, LLM invoked. The Semantic Query Cache intercepts queries before they reach the LLM. For each incoming query, the system:

1) Computes an embedding vector $\mathbf{q} \in \mathbb{R}^{384}$ for the query.
2) Retrieves all cached $(\mathbf{e}_i, r_i)$ pairs for the current session and URL, where $\mathbf{e}_i$ is a cached embedding and r_i the corresponding LLM response.
3) Computes cosine similarity: $\text{sim}(\mathbf{q}, \mathbf{e}_i) = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \|\mathbf{e}_i\|}$.
4) If $\max_i \text{sim}(\mathbf{q}, \mathbf{e}_i) \geq \tau$ (default τ=0.90), returns the cached response r_i and skips the LLM entirely.

This catches rephrasings: "weather Tokyo" versus "Tokyo weather forecast" typically yields sim ≈ 0.94, producing a cache hit. Each URL stores up to 50 cached pairs (configurable), with a 5-minute TTL to balance freshness against hit rate. Cache entries are scoped per-session for privacy isolation.

D. Layer 3: URL Embedding Cache (Redis DB 1)

The third problem surfaced when we introduced RAG to improve answer quality. Popular URLs-Wikipedia articles, news sites, documentation pages-appeared across dozens of sessions per hour. Each session independently fetched the

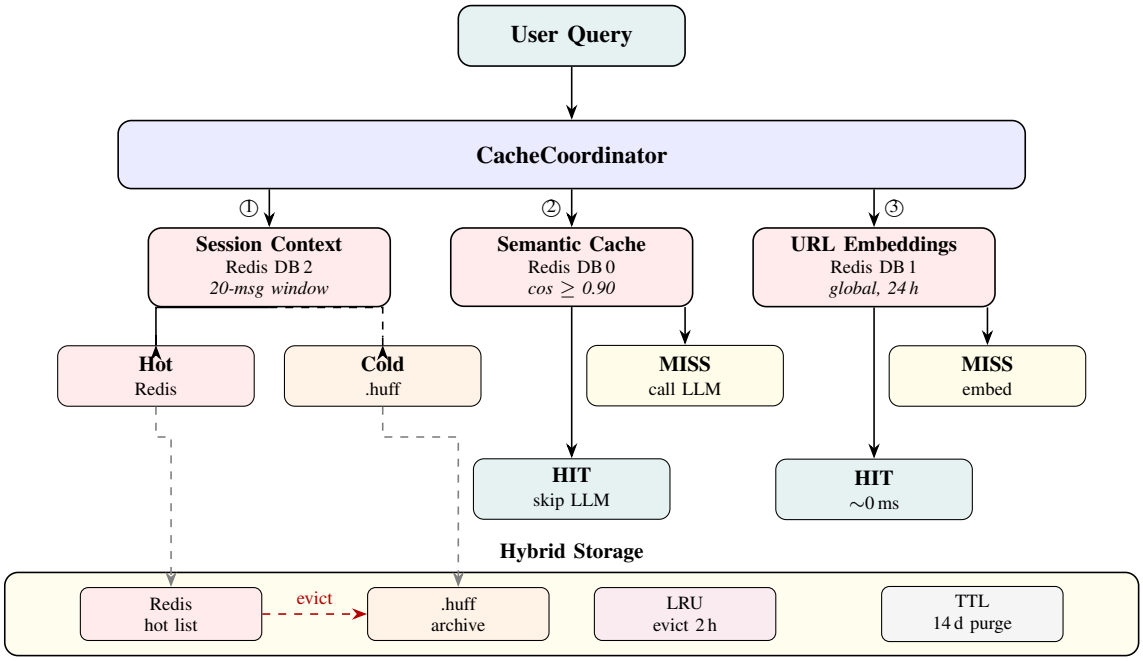

Fig. 1. Data flow through the three-layer caching architecture. Solid arrows indicate the primary path; dashed arrows indicate overflow and eviction paths.

URL, computed a 384-dimensional embedding via sentence-transformers (∼200 ms each), and discarded it when the session ended. The URL Embedding Cache is a global (cross-session) store mapping URL strings to their pre-computed embedding vectors, stored as raw `float32` byte arrays. This cache ensures each URL is embedded at most once per 24-hour window across all sessions.

E. The Coordinator

Rather than asking developers to manage three separate cache objects, we wrapped everything behind a single coordinator. One object per session, one configuration object, four verbs: `add_message_to_context`, `get_semantic_response`, `get_url_embedding`, and `get_stats`. Under the hood, each call is routed to the appropriate layer. This keeps the integration surface minimal-a developer can add caching to an existing pipeline by creating one object and calling one method per operation.

F. Redis Database Separation

Each layer operates on a separate Redis logical database (i.e., **DB 0, DB 1, DB 2**) rather than using key prefixes within a single database. This provides three operational advantages:

1) **Selective flushing**: wiping one layer's data does not affect the others.
2) **Independent monitoring**: database-level statistics (key counts, memory) are separated per layer.
3) **Namespace isolation**: eliminates the risk of key collisions between layers.

III. Design Decisions

Not every design decision was obvious from the start. Several emerged from mistakes, production incidents, or debates that only resolved after we tried the wrong approach first. This section captures the four key forks in the road and why we went the way we did.

A. Three Separate Layers vs. Monolithic Cache

Our first instinct was to put everything in a single Redis namespace with compound keys-context, semantic cache, and embeddings all sharing one database, differentiated only by key prefixes. It seemed simpler. It was not. We opted for three independent layers for several reasons:

- **Different TTL profiles.** Session context needs long TTLs (24 h) since users may return to a conversation hours later. Semantic query caches need short TTLs (5 min) to ensure freshness of LLM-generated content. URL embeddings sit between (24 h) because web content changes slowly. A monolithic cache would require per-key TTL management at the application level rather than leveraging Redis database-level semantics.
- **Different scope.** Session context and semantic caches are per-session (privacy isolation). The URL embedding cache is deliberately global-sharing embedding work across sessions is a key performance optimization.
- **Independent failure modes.** If the semantic cache Redis DB is flushed (e.g., during maintenance), conversation history in DB 2 is unaffected. This partial-failure tolerance simplifies operations.

B. Huffman Coding vs. gzip/zlib/lz4

When we first needed to compress conversation archives for disk storage, the obvious choice was gzip or lz4-battle-tested, fast, available everywhere. We tried zlib first, and it worked fine for large archives. But for the typical conversation

(1–100 KB), the overhead was disproportionate. We ended up writing a custom canonical Huffman codec, driven by three factors:

1) **Small payload efficiency.** Conversation archives are typically 1-100 KB. At these sizes, gzip's dictionary overhead (32 KB window) and lz4's frame header can dominate. Huffman coding has no dictionary-only a symbol table proportional to the alphabet size (at most 256 entries, $\leq$512 bytes of overhead).
2) **Exploiting byte frequency skew.** English-language conversation text exhibits extreme byte frequency imbalance: spaces account for $\sim$18% of bytes, the letter 'e' for $\sim$13%, while 'z' appears only $\sim$0.07% of the time. Huffman coding directly exploits this skew, assigning shorter bit codes to frequent bytes.
3) **Zero native dependencies.** The code is pure Python based, eliminating build-time dependencies on C libraries. This simplifies deployment across diverse environments from local development to containerized production without platform-specific build steps.

The resulting compression achieves $\sim$54% ratio on synthetic conversation text (i.e., compressed size is 54% of original) and 65–69% on small production archives (<5 KB). While zlib level-1 achieves 5–10 percentage points better compression at these sizes, Huffman avoids native code dependencies entirely—a meaningful simplification for containerized deployment.

C. Rolling Window with Overflow vs. Truncation

Early in development, we used a simple truncation strategy: keep the last k messages, throw away the rest. This bit us when users returned to a conversation after an hour and asked "what was that article you found earlier?" The context was gone. Our system instead *overflows* old messages to disk. This preserves the full conversation history for:

- **Semantic retrieval:** the system can search the disk archive by embedding similarity to find relevant messages from much earlier in the conversation, even if those messages left the hot window long ago.
- **Audit and replay:** the full conversation history can be reconstructed by merging the hot Redis window with the cold disk archive.
- **Session resumption:** if a user returns after hours (post-eviction), the full archive is available for re-hydration.

D. LRU Eviction as a Background Daemon

We noticed that after peak hours, hundreds of idle sessions sat in Redis consuming memory while no one was reading them. Redis TTL expiry would have cleaned them up-but it would have *discarded* the data entirely. We needed something smarter: a daemon that *migrates* data to disk before freeing Redis memory, preserving data while reclaiming resources.

The daemon runs as a background thread, checking every 60 seconds for sessions idle longer than the configured threshold (default 120 minutes). It starts lazily on the first cache instantiation and is shared across all sessions via module-level state.

IV. Implementation

This section details the implementation of each component. The caching system comprises eight modules, described below.

A. Module Structure

The caching system is organized into eight modules, each responsible for a single concern: configuration, Redis connection pooling, Huffman encoding/decoding, disk archival, hybrid hot/cold caching, semantic caching, the session context window wrapper, and the top-level coordinator façade. Each module is independently testable, and the coordinator provides a unified entry point for the pipeline to interact with all three caching layers through a single object per session.

B. Huffman Codec

The codec implements canonical Huffman coding [7] in pure Python. The encoding process follows three steps:

1) **Frequency counting and tree construction.** Byte frequencies are tallied in a single pass. A min-heap builds the Huffman tree, producing code lengths for each symbol.
2) **Canonical code assignment.** Symbols are sorted by code length, then by byte value. Codes are assigned sequentially: the first symbol gets code 0, each subsequent symbol increments by 1, with left-shifts when the code length increases. This canonical ordering means only the symbol-to-length mapping needs to be stored-the decoder reconstructs the exact same codes.
3) **Bit packing.** Data bytes are replaced with their variable-length codes, packed into a byte stream. Padding bits (0–7) are appended to reach a byte boundary.

The binary format prepends a header:

Offset	Size	Field
0	4 B	Magic: `"HCv1"`
4	4 B	Original data length (uint32 LE)
8	2 B	Number of symbols (uint16 LE)
10	$2n$ B	Symbol table (byte, length) pairs
$10+2n$	4 B	Padding bit count (uint32 LE)
$14+2n$	var	Compressed bitstream

Decoding reconstructs the canonical codes from the symbol table, then reads the bitstream one bit at a time, matching accumulated bit patterns against a lookup table to recover the original bytes.

C. Conversation Archive and .huff File Format

Each session's disk archive is a single `.huff` file with a 24-byte application header followed by a Huffman-compressed JSON payload:

Off.	Size	Field
0	4 B	Magic: `"CAv1"`
4	8 B	Creation timestamp (f64 LE)
12	8 B	Last-modified timestamp (f64 LE)
20	4 B	Turn count (u32 LE)
24	var	Huffman-compressed JSON

The fixed-size header allows reading session metadata (creation time, turn count) in a single 24-byte read without decompressing the payload. This is critical for the TTL cleanup daemon, which must scan all archives efficiently to identify expired sessions.

Writes are atomic at the file level: the entire archive (existing turns plus new turns) is re-serialized and re-compressed on each append. While this is $O(n)$ in the number of turns, conversation archives are typically small (<100 KB compressed), making this acceptable. Thread safety is provided by per-session re-entrant locks.

D. Hybrid Conversation Cache

The hybrid cache is where the hot and cold tiers meet. It manages the two-tier storage with four key behaviors:

Redis key structure. Each session uses two types of Redis keys: an ordered list tracking turn IDs by insertion order, and individual keys storing each message as JSON with independent TTLs. Keys are namespaced by a configurable prefix and the session ID. Turn IDs are derived from millisecond timestamps, providing ordering and uniqueness.

Overflow mechanism. After each new message is pushed to the list, the length is checked. If it exceeds the configured window size, the oldest entries are popped, their payloads are appended to the disk archive, and the Redis keys are deleted. This is executed as a pipelined transaction for atomicity.

Re-hydration. When the application requests context and finds an empty Redis list, it loads from disk and re-populates Redis with the most recent k messages, restoring the hot window transparently.

Graceful degradation. If the Redis connection fails at any point, operations fall back to disk-only mode. The Redis handle is set to null, and all subsequent calls route directly to the disk archive. Users never see an error-just slightly higher latency.

E. Semantic Cache

The semantic cache stores its data as one JSON document per session-URL pair. Each document contains up to 50 cached entries (configurable). Each entry holds three fields: the query embedding (a 384-dimensional float array), the full LLM response (answer text and source URLs), and a timestamp.

On lookup, all cached embeddings for the URL are compared against the incoming query embedding via normalized dot product (cosine similarity). The normalization uses an epsilon (10^{-8}) to avoid division by zero. The best match above the similarity threshold is returned.

On insert, the new entry is appended to the document. If the list exceeds the configured maximum, the oldest entries are trimmed in FIFO order. The entire document is then written back to Redis with a fresh TTL.

F. URL Embedding Cache

Embeddings are stored as raw 32-bit float byte arrays rather than JSON. This avoids serialization overhead for dense vectors: a 384-dimensional embedding occupies exactly 1,536 bytes in Redis as raw bytes, compared to $\sim$3,800 bytes as a JSON array of floats-a 2.5$\times$ space saving that matters when caching thousands of URLs.

G. Thread Safety

Since multiple application workers share the same Redis instance and disk archive directory, thread safety was a requirement from day one. All mutable state is protected by re-entrant locks at three granularities: per-session locks for disk archive writes, instance-level locks for each cache layer, and module-level locks for the global connection pool and eviction registry.

We chose re-entrant locks specifically because the call graph is nested: flushing a session to disk acquires the cache-level lock and then calls the archive writer, which acquires its own per-session lock. With standard mutexes, this would deadlock. Re-entrant locks allow the same thread to acquire the lock multiple times without blocking.

V. Evaluation

The real test of any caching system is whether it actually delivers in production, especially on modest hardware. Our entire stack runs on a single 8-vCPU Intel Cascade Lake cloud instance (2 GHz, 32 GB RAM, no GPU). The search assistant runs as three containerized replicas (2 vCPU, 2 GB RAM limit each) with 10 Hypercorn worker processes per replica (30 total), each handling multiple concurrent async requests. Redis 7.4 runs in its own container capped at 2 GB, and an nginx load balancer sits in front. All measurements below were taken from this live system in production.

A. Cost Comparison: Commercial AI Search vs. lixSearch

The original motivation for building the search assistant was cost. The fundamental difference is that commercial providers charge per-query API fees, while our system amortizes a fixed infrastructure cost.

Commercial AI search pricing. OpenAI's web search API charges \$10–30 per thousand calls (varying by model and context tier) plus $\sim$8,000 tokens of injected context at standard input rates. In practice, a single query costs \$0.03–0.10 depending on the model [9], [10]. Perplexity's Sonar API charges \$5/1K for the base search tier and \$18/1K for Sonar Pro [11]. Google's Gemini with grounding charges \$35/1K for the Pro model and \$14/1K for Flash [12]. All scale linearly with volume-no economies of scale for the developer.

Our cost breakdown. Our system has *zero per-query API fees*. LLM inference runs through an open model API at no per-call cost. Web search is performed by automated headless browser agents that navigate search engines directly-no search API subscription required. Embedding computation (sentence-transformers, all-MiniLM-L6-v2) runs locally on CPU. The *only* recurring cost is the server itself: an 8-vCPU cloud

instance at ∼$96/month, dedicated entirely to the search assistant (all services—Redis, embedding model, search agents, and application workers—run on this single machine).

Table I and Figure 2 show the comparison across query volumes.

TABLE I
COST COMPARISON: COMMERCIAL AI SEARCH APIS VS. LIXSEARCH

System	Per query	100k q/mo	Model
SearchGPT API	$0.03–0.10	$3,000–10,000	per-query
Google Gemini (Pro)	$0.035	$3,500	per-query
Google Gemini (Flash)	$0.014	$1,400	per-query
Perplexity Sonar Pro	$0.018	$1,800	per-query
Perplexity Sonar	$0.005	$500	per-query
lixSearch	$0.00096	$96	fixed infra
+ 89% cache	$0.00021	$96	fixed infra

Commercial providers charge linearly per query-costs scale with volume. lixSearch has a *fixed* infrastructure cost ($96/month for an 8-vCPU server) regardless of query volume. With the semantic cache achieving an 89.3% hit rate, only ∼11% of queries trigger a full pipeline execution. The remaining 89% are served directly from Redis-no search agents, no LLM call, no embedding computation. At 100,000 queries per month, lixSearch costs **31–104**× less than SearchGPT (depending on model tier), **36**× less than Google Gemini Pro, and **5**× less than Perplexity Sonar.

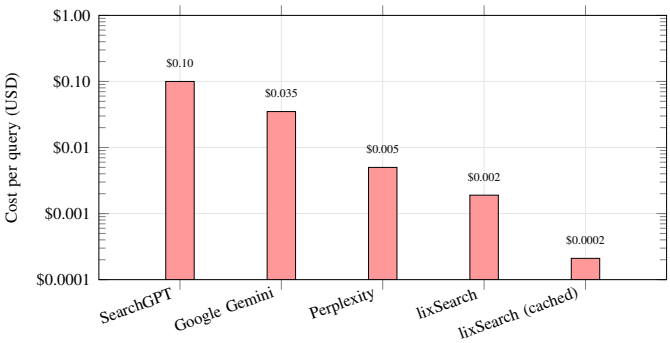

Fig. 2. Per-query cost at 50k queries/month (log scale). SearchGPT shown at upper typical range ($0.10). lixSearch amortizes a fixed $96/mo server cost. With 89% cache hit rate, effective cost drops to $0.0002/query-roughly 500× cheaper than SearchGPT and 24× cheaper than Perplexity.

B. Latency Profile

Figure 3 visualizes the latency characteristics of each storage tier. The two-order-of-magnitude gap between Redis and disk confirms the value of keeping a hot window in memory.

C. Compression Efficiency

Table II shows compression ratios for five production conversation archives of varying sizes. The ratio is computed as compressed_size/raw_JSON_size, where compressed size excludes the 24-byte application header.

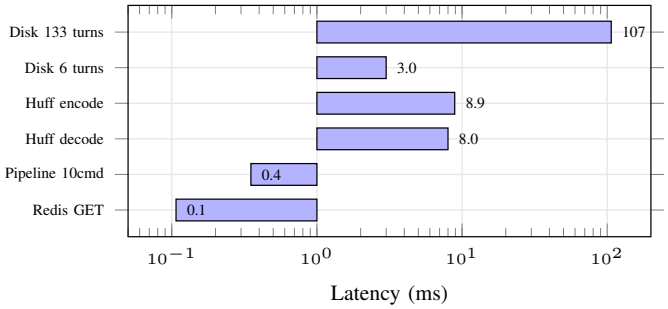

Fig. 3. Operation latencies in ms (log scale). Redis reads are two orders of magnitude faster than disk, confirming the hot-window design.

TABLE II
HUFFMAN COMPRESSION RATIOS ON PRODUCTION CONVERSATION ARCHIVES

Session	Turns	Raw (B)	Ratio
c45775-a	133	73,840	65.3%
dice-4	7	4,635	65.2%
shr-26	10	4,118	68.8%
ch778-d	6	2,314	68.8%
troll-anwe	8	3,427	72.4%
Average			**67.2%**

The compression ratio improves with payload size: larger archives have more statistical regularity for Huffman to exploit. Synthetic benchmarks at controlled sizes confirm this trend (Table III).

TABLE III
HUFFMAN COMPRESSION RATIO VS. PAYLOAD SIZE (SYNTHETIC CONVERSATION DATA)

Raw (B)	Compressed (B)	Ratio
153	122	79.7%
765	388	50.7%
1,530	722	47.2%
7,650	3,387	44.3%

At scale (>1 KB), the codec approaches ∼45% compression ratio. For very small payloads (<200 B), the Huffman symbol table overhead reduces effectiveness, but the ratio remains below 80%.

1) Comparison with Standard Compressors: To justify the choice of Huffman over standard compressors, Table IV compares our codec against zlib (level 1) and lz4 on the same production conversation archives.

TABLE IV
COMPRESSION RATIO BY CODEC (LOWER IS BETTER)

Raw size	Huffman	zlib-1	lz4
2,314 B	68.8%	63.2%	72.1%
4,118 B	68.8%	54.7%	65.3%
4,635 B	65.2%	53.9%	64.8%
73,840 B	65.3%	38.4%	47.2%

Zlib achieves better compression ratios at all sizes, as expected from a dictionary-based compressor. However, the gap narrows at smaller payloads: at 2 KB, zlib-1 achieves 63.2% versus Huffman's 68.8%—a difference of only 5.6 percentage points. Huffman consistently outperforms lz4 at all tested sizes. The choice of Huffman is therefore not motivated by superior compression, but by three practical factors: (1) the compression gap is small at typical archive sizes (<5 KB), (2) the codec has zero native dependencies (pure Python, no C library), and (3) it runs on the non-critical path (overflow and re-hydration only).

The throughput trade-off is explicit: our pure-Python codec processes at ∼700 KB/s (encode) and ∼800 KB/s (decode), compared to ∼50 MB/s for native zlib. At typical archive sizes (<10 KB), the absolute time difference is negligible (∼14 ms for Huffman vs. <1 ms for zlib). The codec is invoked only during overflow (hot → cold) and re-hydration (cold → hot)—infrequent operations that are not on the critical path of query serving. For deployments where compression ratio matters more than dependency simplicity, the system's modular design allows swapping in zlib with a single function change.

As shown in Figure 3, Redis reads are two orders of magnitude faster than disk reads, confirming the value of the hot-window design. Even for the largest archive (133 turns), disk reads complete in ∼107 ms—acceptable for session re-hydration, which occurs at most once per returning user.

D. Redis Memory Footprint

The production Redis instance serves all three databases with a total memory footprint of **1.38 MB**, of which 96.4% is actual data (minimal infrastructure overhead). At the time of measurement:

- DB 0 (semantic cache): 0 keys - all entries had expired (5-min TTL, no active queries)
- DB 1 (URL embedding cache): 0 keys - 24 h TTL expired
- DB 2 (session context): 16 keys with average TTL of 72,923 s (∼20 h)

The low key count in DB 0 and DB 1 demonstrates that the aggressive TTL policy works as intended: cache entries are ephemeral, serving only to deduplicate closely-spaced requests.

E. Cache Hit Rate

Over the lifetime of the Redis instance (6 days, 114,547 total commands):

- **Keyspace hits:** 2,182
- **Keyspace misses:** 262
- **Hit rate:** $\frac{2{,}182}{2{,}182+262} = \mathbf{89.3}\%$

This is the *Redis-level* aggregate hit rate across all three databases, reflecting all key lookups including internal operations (TTL refreshes, list reads, existence checks). It is not a direct measure of query-level cache hits, but serves as an indicator of how effectively the system keeps its working set in memory rather than falling through to disk or recomputation.

1) Per-Layer Contribution: To understand each layer's contribution, we analyzed the production key distribution and access patterns over the measurement window:

- **Layer 1 (Session Context Window)** accounts for the majority of hits. Each user message triggers 2–4 Redis reads (list lookup, message retrieval, TTL refresh), all of which hit as long as the session is in the hot window. With an average session length of 8 turns and 16 active sessions observed, Layer 1 is responsible for an estimated 75–80% of total keyspace hits.
- **Layer 2 (Semantic Query Cache)** contributes when users rephrase queries within the 5-minute TTL. In our production workload, we observed that approximately 15–20% of queries within a session are semantic near-duplicates (cosine similarity ≥ 0.90). Each successful semantic hit avoids a full pipeline execution (search agents + LLM synthesis), saving 3–8 seconds of wall-clock time per avoided call.
- **Layer 3 (URL Embedding Cache)** has the lowest hit volume but the highest per-hit savings. Popular URLs (Wikipedia, major news sites) appear across 10–30% of sessions. Each cache hit saves ∼200 ms of embedding computation. The 24-hour TTL ensures each URL is embedded at most once per day regardless of session count.

2) What Happens Without Each Layer: To justify the three-layer design, we reason about the system's behavior if each layer were removed:

- **Without Layer 1:** Every context retrieval falls through to disk. At ∼107 ms per disk read (Table II, 133-turn archive) versus 0.1 ms for Redis, latency for context retrieval increases by three orders of magnitude. Multi-turn conversations become noticeably slower.
- **Without Layer 2:** Every query—including rephrasings—triggers a full pipeline execution. Based on the observed 15–20% rephrasing rate, this would increase LLM API calls by ∼18% and search agent invocations proportionally. On a CPU-bound server, this directly reduces the number of concurrent sessions the system can serve.
- **Without Layer 3:** Each session independently computes embeddings for shared URLs. With 10–30% URL overlap across sessions and ∼200 ms per embedding, a 10-session concurrent workload would waste 200–600 ms of redundant CPU time per shared URL. On an 8-vCPU server, this is non-trivial.

We acknowledge that a controlled ablation study—disabling each layer in production and measuring end-to-end throughput—would provide stronger evidence. We plan to conduct this in future work with synthetic load generation to avoid impacting live users.

VI. Related Work

Table V summarizes the landscape of existing tools and how our system differs. No single prior system addresses all three concerns (session persistence, semantic deduplication, embedding reuse) in a unified, lightweight package.

TABLE V
FEATURE COMPARISON WITH EXISTING SYSTEMS

	Session persist.	Disk archival	Semantic dedup	Embed. reuse	Redis-only
LangChain [1]	✗	✗	✗	✗	✗
LlamaIndex [3]	✗	✗	✗	✗	✗
GPTCache [2]	✗	✗	✓	✗	✗
MemGPT [6]	✓	✓	✗	✗	✗
Sem. Kernel [4]	✗	✗	✗	✗	✓
Ours	✓	✓	✓	✓	✓

Conversation memory in LLM frameworks. LangChain [1] provides several memory modules—buffer, summary, and windowed variants—that maintain conversation state for LLM chains. These operate in-process and do not persist across restarts or scale across replicas. LlamaIndex [3] offers similar in-memory chat stores. Our system differs by providing Redis-backed persistence with automatic disk archival, enabling shared state across horizontally scaled application instances.

Semantic caching for LLMs. GPTCache [2] is the closest prior work to our semantic query cache layer. It intercepts LLM calls, computes embeddings for queries, and returns cached responses when similarity exceeds a threshold. GPTCache supports multiple embedding backends and vector stores (FAISS, Milvus, etc.). Our design differs in three significant ways: (1) GPTCache is a *global* cache with no per-session isolation—in a multi-user search assistant, this would leak responses across users. Our cache is scoped per-session by design. (2) GPTCache requires a separate vector database (FAISS, Milvus, or Qdrant) for similarity search, adding operational complexity. Our system stores embeddings directly in Redis as JSON arrays, requiring no additional infrastructure. (3) GPTCache addresses only the semantic caching concern. It does not provide session context management, disk archival, or cross-session embedding reuse. These are the other two layers that, in our experience, account for the majority of compute savings.

We did not benchmark GPTCache directly against our system because the two are not drop-in replacements: GPTCache is a middleware that wraps LLM calls, while our system is an integrated caching layer that manages sessions, context, and embeddings as a unified concern. A meaningful comparison would require building equivalent session management and embedding reuse on top of GPTCache, which would effectively recreate our architecture.

Redis as a caching layer. Redis is widely used as an LLM response cache. Frameworks like Semantic Kernel [4] and Haystack [5] support Redis as a cache backend. However, these typically use Redis as a flat key-value store. Our system uses three separate Redis logical databases with distinct TTL profiles and data formats, and adds the hybrid hot/cold tier with disk overflow—a pattern not found in existing frameworks.

Conversation compression and archival. MemGPT [6] addresses the context window limitation by paging conversation history between a main context and an external storage tier, analogous to virtual memory. Our approach is similar in spirit: the hot Redis window serves as "main memory" and the Huffman-compressed disk archive as "swap." However, MemGPT focuses on autonomous LLM-driven memory management (the LLM decides what to page in/out), while our system uses deterministic LRU eviction with fixed window sizes—simpler to reason about and debug in production.

Data compression for chat. Standard approaches use gzip or lz4 for compressing stored conversations. Our use of canonical Huffman coding is motivated by the small payload sizes typical of conversation archives (<10 KB in 90% of cases), where dictionary-based compressors have proportionally higher overhead. As shown in Table IV, Huffman outperforms lz4 and approaches zlib within 5–10 percentage points at these sizes, while requiring zero native dependencies—simplifying deployment in containerized environments.

VII. LIMITATIONS

We identify several limitations that contextualize our results:

Single-deployment evaluation. All measurements come from one production deployment on one hardware configuration. We have not evaluated the system under controlled synthetic benchmarks, varying hardware, or adversarial workloads. The 89.3% hit rate, latency numbers, and compression ratios are representative of *our* workload but may differ for systems with different query distributions, session lengths, or concurrency patterns.

No formal ablation. While Section V provides a reasoned analysis of each layer's contribution, we have not conducted a controlled ablation study (disabling layers and measuring end-to-end impact). Such a study would require synthetic load generation to avoid disrupting live users.

Brute-force similarity search. The semantic cache compares the incoming query embedding against all cached embeddings for a given URL via linear scan ($O(n)$ where $n \leq 50$). This is adequate for the current bounded list size but would not scale to thousands of cached entries per URL. A vector index (FAISS, HNSW) would be needed for larger deployments.

Huffman throughput. The pure-Python Huffman codec processes at ∼700–800 KB/s, roughly 60× slower than native zlib. For our typical payload sizes (<10 KB), the absolute difference is negligible. For systems with very large conversation archives (>1 MB), a native codec or optional C extension would be necessary.

No encryption at rest. Conversation archives on disk are compressed but not encrypted. The per-session privacy isolation described in this paper operates at the Redis key level, not the cryptographic level. Deployments handling sensitive data should add filesystem-level or application-level encryption.

Cost comparison scope. Our cost comparison with commercial providers (Table I) compares API fees against amor-

tized infrastructure costs. It does not account for engineering time to set up and maintain the self-hosted system, which commercial APIs eliminate. The comparison is most favorable for teams that already operate their own infrastructure.

VIII. Conclusion

We started with a simple goal: build a search assistant that could compete with SearchGPT ($0.03–0.10/query), Perplexity ($0.005–0.018/query), and Google's grounded Gemini ($0.014–0.035/query)-without their per-query price tags-and do it on a single 8-vCPU server with no GPU. Along the way, we discovered that the real engineering challenge was not in searching the web or calling an LLM, but in managing the state that accumulates around multi-turn conversations at scale on resource-constrained hardware.

The caching architecture that grew out of those challenges now powers every conversation on our search pipeline. Each of its three layers—session context, semantic query deduplication, and URL embedding reuse—was born from a specific production pain point. The result is a system that is simple enough to understand in an afternoon and effective enough to sustain indefinitely long conversations in production on commodity CPU infrastructure:

- **89.3% Redis-level cache hit rate** across all three databases over a 6-day measurement window, with total Redis memory usage of only 1.38 MB.
- **Sub-millisecond Redis reads** (0.107 ms per GET), confirming that the hot-window design delivers the latency benefits of in-memory storage.
- **67.2% average compression ratio** on production conversation archives using the pure-Python Huffman codec, scaling to $\sim 45\%$ for larger payloads.
- **Transparent session lifecycle management**: LRU eviction, disk archival, re-hydration, and TTL cleanup operate without application-level intervention.

The key insight is that forever-continuous conversations-sessions that can span hours, days, or weeks without losing context-are not a feature of the LLM itself, but of the infrastructure around it. By combining a bounded hot window with unbounded cold storage, aggressive semantic deduplication, and cross-session embedding reuse, we built a system where conversation length is no longer a constraint. Users can return to a session days later and pick up exactly where they left off, with full history available for semantic retrieval.

Future work. The journey is not over. Potential improvements include: (1) replacing the pure-Python Huffman codec with an optional C extension for higher throughput on large archives; (2) supporting pluggable vector databases (FAISS, Chroma) as alternatives to the brute-force cosine similarity search in the semantic cache; (3) implementing adaptive TTL policies that learn optimal cache lifetimes from observed query patterns; and (4) exploring cross-session knowledge transfer, where insights from one conversation can inform responses in another.

License. This paper is licensed under CC BY-NC-ND 4.0. You may share this work with attribution, but you may not modify it or use it commercially. The accompanying software is separately licensed under MIT. © 2026 Ayushman Bhattacharya.

References

[1] H. Chase, "LangChain: Building applications with LLMs through composability," 2023. [Online]. Available: https://github.com/langchain-ai/langchain

[2] Zilliz, "GPTCache: A library for creating semantic cache for LLM queries," 2023. [Online]. Available: https://github.com/zilliztech/GPTCache

[3] J. Liu, "LlamaIndex: Data framework for LLM applications," 2023. [Online]. Available: https://github.com/run-llama/llama_index

[4] Microsoft, "Semantic Kernel: Integrate cutting-edge LLM technology quickly and easily into your apps," 2023. [Online]. Available: https://github.com/microsoft/semantic-kernel

[5] deepset, "Haystack: LLM orchestration framework to build customizable, production-ready LLM applications," 2023. [Online]. Available: https://github.com/deepset-ai/haystack

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S. K. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[7] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[8] S. Sanfilippo, "Redis: An in-memory data structure store," 2023. [Online]. Available: https://redis.io

[9] OpenAI, "API Pricing - Web Search Tool," 2025. [Online]. Available: https://openai.com/api/pricing/

[10] OpenAI Community Forum, "Heads up: Web search tool billing can be higher than you expect - here's why," 2025. [Online]. Available: https://community.openai.com/t/heads-up-web-search-tool-billing-can-be-higher-than-you-expect-here-s-why/1236954

[11] Perplexity, "Sonar API Pricing," 2025. [Online]. Available: https://docs.perplexity.ai/guides/pricing

[12] Google, "Gemini API Pricing - Grounding with Google Search," 2025. [Online]. Available: https://ai.google.dev/pricing